

# Scribblez: Toward Superhuman Scrabble AI

A Technical Design Document

David Shin

May 29, 2026

Draft v0.1

## Abstract

We present the design for a Scrabble AI that aims to surpass current state-of-the-art engines (e.g., Macondo) by addressing two fundamental weaknesses: context-blind equity leave estimation and naive opponent rack inference. Our architecture combines neural network evaluation with combinatorial move generation (GADDAG), a particle-filter-based Bayesian belief system over hidden racks, and a unified Q/V network for both move selection and position evaluation. We maintain a *public belief state*—a joint distribution over both players’ hidden racks from the perspective of an outside observer—and use a learned encoder/decoder framework with posterior refinement to produce accurate, context-aware rack samples for Monte Carlo simulation.

## Contents

|          |                                                    |          |
|----------|----------------------------------------------------|----------|
| <b>1</b> | <b>Introduction and Motivation</b>                 | <b>3</b> |
| <b>2</b> | <b>System Overview</b>                             | <b>3</b> |
| <b>3</b> | <b>Public Belief System</b>                        | <b>4</b> |
| 3.1      | Motivation . . . . .                               | 4        |
| 3.2      | Belief Encoder/Decoder Architecture . . . . .      | 5        |
| 3.3      | Bayesian Posterior Refinement . . . . .            | 5        |
| 3.4      | Rejection Traces as Information . . . . .          | 6        |
| 3.5      | Iterative Particle Generation . . . . .            | 6        |
| 3.6      | Particle Filter Across Turns . . . . .             | 7        |
| <b>4</b> | <b>Move Evaluation: Unified Q/V Network</b>        | <b>7</b> |
| 4.1      | Architecture . . . . .                             | 7        |
| 4.2      | Move Encoding . . . . .                            | 7        |
| 4.3      | Integration of Belief State . . . . .              | 8        |
| 4.4      | Handling Large Move Sets and Blank Tiles . . . . . | 8        |
| <b>5</b> | <b>Search Procedure</b>                            | <b>8</b> |
| 5.1      | Root Node: Move Selection . . . . .                | 8        |
| 5.2      | Monte Carlo Simulation . . . . .                   | 8        |
| 5.3      | Belief State Updates During Simulation . . . . .   | 9        |

|          |                                            |           |
|----------|--------------------------------------------|-----------|
| <b>6</b> | <b>Training</b>                            | <b>9</b>  |
| 6.1      | Self-Play Data Generation . . . . .        | 9         |
| 6.2      | Belief System Training . . . . .           | 9         |
| 6.3      | Q-Head Training . . . . .                  | 10        |
| 6.4      | V-Head Training . . . . .                  | 10        |
| 6.5      | Training Loop . . . . .                    | 10        |
| <b>7</b> | <b>Summary of Design Principles</b>        | <b>10</b> |
| <b>8</b> | <b>Future Directions</b>                   | <b>11</b> |
| 8.1      | Search-Derived Knowledge Buffers . . . . . | 11        |

# 1 Introduction and Motivation

Current state-of-the-art Scrabble engines, such as Macondo, operate by generating candidate moves and evaluating them through Monte Carlo simulation. While effective, these engines rely on two key heuristics that we believe represent their primary sources of weakness:

**Context-blind equity leave estimation.** After a move is played, the remaining tiles on the rack (the *leave*) are assigned a value from a precomputed lookup table. This leave value depends only on the tiles themselves and is independent of the board state, score differential, tiles remaining in the bag, or any other game context. The same leave (e.g., AEINR) receives the same valuation whether the board is wide open or locked down, whether the player is ahead by 100 points or behind by 50, and whether the bag contains 80 tiles or 5.

**Naive opponent rack inference.** When sampling opponent racks for simulation, Macondo uses a rejection sampling heuristic: sample a random rack from the unseen tile pool, and accept it if the opponent’s most recent move would have been “plausible” given that rack. This approach has two critical limitations:

1. **Myopia:** It conditions only on the opponent’s last move, discarding all earlier history. A low-value S play two turns ago—strong evidence of a duplicate S—is ignored entirely.
2. **Sparsity:** After moves involving a small number of observed tiles (or no observed tiles in the case of an exchange), most sampled moves will get rejected as implausible, so the acceptance filter has little effect and sampling degenerates to near-uniform over unseen tiles<sup>1</sup>.

Our design addresses both weaknesses by introducing neural network components that condition on the full game state, combined with Bayesian inference machinery that refines neural predictions using combinatorial analysis.

## 2 System Overview

The system consists of the following major components:

1. **GADDAG move generator:** Enumerates all legal moves from a given board state and rack.
2. **Public belief system:** Maintains a joint probability distribution over both players’ hidden racks, conditioned on all public information.
3. **Shared board encoder:** A neural network that produces a rich representation of the current board state.
4. **Q-head:** Accepts the board encoding and a variable-length set of move encodings; outputs a value estimate for each candidate move in a single forward pass.
5. **V-head:** Accepts a board state encoding and outputs a scalar position value.
6. **Belief encoder/decoder:** Produces and samples from a compact latent representation of the rack distribution.
7. **Belief compressor:** A recurrent network that ingests the full trace of particle generation—including rejected candidates and their rejection reasons—and maintains a running belief state.

---

<sup>1</sup>Macondo uniformly samples racks, filtering those that get rejected as implausible. If it fails to produce enough plausible samples within a given budget, it fills the remainder of the sample with unfiltered uniformly sampled racks. This unfiltered filler is what leads to the near-uniform degeneracy.

8. **Monte Carlo simulation engine:** Uses all of the above to evaluate candidate moves via rollout.

Figure 1 provides a high-level view of the architecture and data flow.

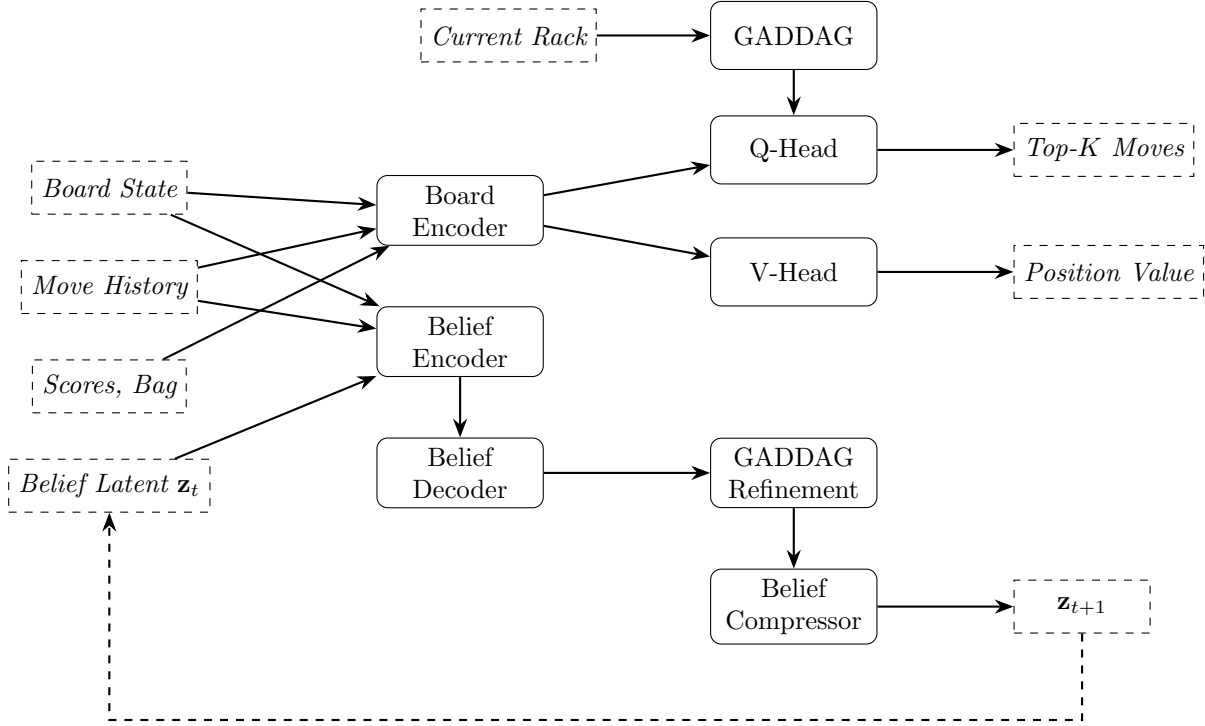


Figure 1: High-level architecture. Solid arrows indicate data flow within a single evaluation. The dashed arrow indicates the belief latent feedback loop across turns.

### 3 Public Belief System

#### 3.1 Motivation

In Scrabble, each player’s rack is private information. Reasoning about the opponent’s rack is critical for strong play, but equally important is modeling what the *opponent* infers about *our* rack. Following the approach used in poker AI research (e.g., DeepStack, Pluribus), we maintain a **public belief state**: a joint distribution over both players’ racks conditioned on all publicly observable information.

**Definition 1** (Public Belief State). *Let  $h_t$  denote the public history at time  $t$ , consisting of the sequence of all moves played, the visible board state, scores, and the known bag composition. The public belief state is:*

$$b_t(\mathbf{r}_A, \mathbf{r}_B) = P(\mathbf{r}_A, \mathbf{r}_B \mid h_t)$$

where  $\mathbf{r}_A$  and  $\mathbf{r}_B$  are the racks of players  $A$  and  $B$ , respectively.

When it is player  $A$ ’s turn to act,  $A$  knows their own rack  $\mathbf{r}_A$  and can compute the conditional:

$$P(\mathbf{r}_B \mid h_t, \mathbf{r}_A) = \frac{b_t(\mathbf{r}_A, \mathbf{r}_B)}{\sum_{\mathbf{r}'_B} b_t(\mathbf{r}_A, \mathbf{r}'_B)}$$

This gives player  $A$ ’s posterior belief about the opponent’s rack. Crucially, because the public belief is maintained symmetrically, it also encodes what  $B$  would infer about  $A$ ’s rack—enabling reasoning about the opponent’s strategic perspective.

### 3.2 Belief Encoder/Decoder Architecture

The space of possible racks is approximately  $3.2 \times 10^6$ , making it impractical to maintain an explicit probability distribution. Instead, we use a learned encoder/decoder pair with a compact latent representation.

**Encoder.** The belief encoder takes as input:

- The board state (tile placements and premium square locations)
- The full move history (sequence of moves played by both players)
- Game metadata (scores, tiles remaining in bag, turn number)
- The current belief latent  $\mathbf{z}_t$  (from the previous turn’s posterior)

It produces an updated latent  $\mathbf{z}'_t$  that serves as the prior for the current turn.

**Decoder.** The decoder takes the latent  $\mathbf{z}'_t$  plus a random seed and produces a sampled rack. When sampling from player  $A$ ’s perspective,  $A$ ’s known rack is provided as a conditioning input, and the decoder produces samples for  $B$ ’s rack. Efficient sampling requires only a forward pass through the decoder, enabling rapid generation of Monte Carlo samples.

**Training.** The encoder/decoder is trained via self-play, where ground-truth racks are known. The training target is the actual rack held by the player, and the loss encourages the decoder to produce accurate samples given the public history. Importantly, the belief system is trained *independently* of the value head—the latent should faithfully represent the rack distribution without being distorted by value estimation objectives.

### 3.3 Bayesian Posterior Refinement

The neural network’s predictions serve as a *prior* over opponent racks. We refine this prior using combinatorial analysis via the GADDAG:

1. **Sample:** Draw a candidate rack  $\mathbf{r}$  from the decoder.
2. **Reconstruct:** Infer the full rack the opponent held before their last move:  $\mathbf{r}_{\text{full}} = \mathbf{r} \cup \text{tiles\_played}$ .
3. **Analyze:** Using the GADDAG, generate all legal moves for  $\mathbf{r}_{\text{full}}$  on the board state *before* the opponent’s move. Evaluate each legal move using the Q-head.
4. **Accept/Reject:** Assess the plausibility of the opponent’s actual move given the alternatives. If  $\mathbf{r}_{\text{full}}$  would have permitted a dramatically superior move (e.g., a bingo) that the opponent did not play, reject the sample. More formally, assign a plausibility weight:

$$w(\mathbf{r}) \propto P(\text{move}_{\text{observed}} \mid \mathbf{r}_{\text{full}}, \text{board})$$

where the likelihood can be modeled as a softmax over the Q-head’s move values, or a simpler heuristic based on the rank or equity gap of the observed move relative to the best available move.

This two-stage approach leverages the strengths of both components: the neural network captures soft, contextual, history-dependent patterns (e.g., the duplicate-S signal), while the GADDAG

enforces hard combinatorial constraints that the network may miss, especially around rare words unseen during training.

### 3.4 Rejection Traces as Information

A key insight is that *rejected* candidate racks carry as much or more information than accepted ones. When a candidate rack is rejected, the rejection comes with a reason: “if the opponent had held AEINRST, they would have played NASTIER for 83 points instead of their actual 24-point play.” This is a crisp, high-information signal that carves out a region of rack space and explains the shape of the exclusion boundary.

We therefore feed the belief compressor the **full trace** of the particle generation process, including both accepted and rejected candidates. Each trace entry is a tuple:

$$\tau_i = (\mathbf{r}^{(i)}, \text{accepted/rejected}, a_{\text{best}}^{(i)}, Q_{\text{best}}^{(i)}, Q_{\text{observed}}^{(i)})$$

where  $a_{\text{best}}^{(i)}$  is the strongest move available for candidate rack  $\mathbf{r}^{(i)}$ , and  $Q_{\text{best}}^{(i)}$ ,  $Q_{\text{observed}}^{(i)}$  are the Q-values of the best available move and the actually observed move, respectively. The compressor can learn for itself how to weight rejections by severity using these raw features.

This is particularly valuable for handling rare words. If the decoder proposes a rack that would have enabled a bingo through an obscure word the network never encountered in training, the GADDAG catches it during refinement. The rejection trace encodes this knowledge without the compressor needing to know the word itself—it sees the pattern that this combination of tiles, on this board, was rejected because a much stronger play existed.

### 3.5 Iterative Particle Generation

Rather than generating all candidate particles in a single batch, we use an **iterative refinement loop** in which each batch of proposals benefits from the compressor’s updated understanding of the rejection landscape:

1. **Initialize:** Set the compressor state from the previous turn’s belief latent  $\mathbf{z}_t$ .
2. **Propose:** The decoder produces a batch of  $B$  candidate racks, conditioned on the current compressor state.
3. **Evaluate:** GADDAG generates legal moves for each candidate; the Q-head evaluates plausibility. Accept or reject each candidate based on a plausibility threshold.
4. **Update:** Feed the full batch trace (acceptances and rejections with reasons) into the compressor, which updates its internal state.
5. **Repeat:** The decoder produces a new batch conditioned on the *updated* compressor state. Repeat until the accepted particle set reaches the required size  $M$ .

The first batch is essentially sampling from the neural prior. By the second or third batch, the decoder has seen what kinds of racks get rejected and why, and can steer its proposals accordingly. The acceptance rate is expected to climb with each iteration as the decoder learns to avoid implausible regions for the current position.

This is a form of *amortized importance sampling with an adaptive proposal distribution*: rather than using a fixed proposal and reweighting, the proposal itself is iteratively refined based on rejection signal. This is significantly more efficient when the prior and posterior are far apart—precisely the cases where accurate inference matters most.

The compressor in this framework is not a one-shot summarizer but a **recurrent state machine** that processes a stream of evidence. The architecture could be a GRU or transformer that

ingests each batch’s trace and updates a hidden state, with the decoder reading from that state at each iteration.

### 3.6 Particle Filter Across Turns

The belief state evolves across turns via the following steps:

1. **Prediction:** After a move is played, update particles by removing played tiles and sampling new draws from the bag (conditioned on known remaining tiles).
2. **Iterative refinement:** Run the iterative particle generation loop (Section 3.5) to produce a fresh set of  $M$  accepted particles with an updated compressor state.
3. **Feedback:** The compressor’s final state serves as the belief latent  $\mathbf{z}_{t+1}$ , which is provided as input to the belief encoder on the next turn.

The feedback loop creates a virtuous cycle: on turn  $t$ , the decoder produces proposals, many are rejected with informative traces, the compressor encodes the full refinement process into  $\mathbf{z}_{t+1}$ , and on turn  $t + 1$  the decoder receives  $\mathbf{z}_{t+1}$  as input. If the compressor has learned to encode rejection patterns into the latent, the decoder learns to avoid those patterns on subsequent turns—effectively learning from its own mistakes within a single game.

## 4 Move Evaluation: Unified Q/V Network

### 4.1 Architecture

Rather than maintaining separate networks for move selection and position evaluation, we use a single network with a shared board encoder and two heads:

**Board encoder.** Processes the board state (tile placements, premium squares), game metadata (scores, bag composition, turn number), current rack, and the belief latent  $\mathbf{z}_t$  into a rich board representation  $\mathbf{h}$ .

**V-head.** Takes the board representation  $\mathbf{h}$  and outputs a scalar position value  $V(s)$ . Used to evaluate positions at internal simulation nodes.

**Q-head.** Takes the board representation  $\mathbf{h}$  and a variable-length set of move encodings  $\{e(a_1), \dots, e(a_N)\}$ , and outputs a value estimate  $Q(s, a_i)$  for each candidate move in a **single forward pass**. The move encodings are processed via cross-attention against the board representation, allowing the network to learn spatial interactions between move placements and board features.

### 4.2 Move Encoding

Each legal move  $a_i$  is encoded into a feature vector  $e(a_i)$  capturing:

- Tiles played and their board positions
- Immediate score of the move
- The resulting leave (tiles remaining on rack after the move)
- Local board context around the placement (adjacent premium squares, existing tiles)
- Number of tiles played (to distinguish bingo-length plays)

The Q-head processes these via cross-attention with the board encoding:

$$Q(s, a_i) = f(\text{CrossAttn}(e(a_i), \mathbf{h}))$$

where  $f$  projects to a scalar. Because the Q-head operates over a set of variable size, it naturally handles any number of legal moves without a fixed output dimension.

### 4.3 Integration of Belief State

The belief latent  $\mathbf{z}_t$  is provided as an input to the board encoder. This allows both the V-head and Q-head to condition on the posterior-refined belief about hidden racks. For example, the value estimate can reflect “the opponent likely holds a blank with a bingo-friendly leave, so this position is more precarious than the score suggests.”

Because the belief latent is trained independently (Section 3.2), it faithfully represents the rack distribution. The value/Q heads learn to extract strategically relevant information from an honest belief representation, without risk of the belief latent being distorted by value estimation objectives.

### 4.4 Handling Large Move Sets and Blank Tiles

With two blank tiles in hand, the number of legal moves can be very large due to the combinatorial explosion of blank letter designations ( $26^2 = 676$  possible instantiations per physical placement). We manage this through:

**Deduplication.** Moves that produce identical board states (same tiles in same positions, regardless of which physical tile is the blank) are collapsed. This eliminates the majority of blank-induced duplicates.

**Batch evaluation.** After deduplication, the remaining  $N$  moves are encoded and passed to the Q-head in a single forward pass. Typical  $N$  after deduplication is on the order of hundreds, which is a manageable batch size for GPU evaluation.

## 5 Search Procedure

### 5.1 Root Node: Move Selection

At the root of the search:

1. GADDAG generates all legal moves from the current board and rack.
2. After deduplication (Section 4.4), moves are encoded into feature vectors.
3. The Q-head evaluates all moves in a single forward pass.
4. The top  $K$  moves (by Q-value) are selected as candidates for full Monte Carlo simulation.

### 5.2 Monte Carlo Simulation

For each of the  $K$  candidate moves:

1. Sample an opponent rack from the belief system (decoder + posterior refinement).
2. Simulate the game forward for  $D$  plies.
3. At each internal node:
  - GADDAG generates all legal moves.
  - The Q-head selects the top-1 move (greedy play).
  - Rack replenishment is sampled from the bag.
4. The terminal or horizon value is estimated by the V-head.
5. Repeat for  $S$  samples (different opponent rack draws and bag draws).



The simulation-averaged value for each candidate move provides the final ranking for move selection.

### 5.3 Belief State Updates During Simulation

During simulation, the belief state is updated at each ply as new (simulated) moves provide public evidence. The particle filter prediction and refinement steps apply within the simulation, though a lighter-weight version may be used for computational efficiency.

## 6 Training

### 6.1 Self-Play Data Generation

All training data is generated through self-play games. During each game, we record at every position:

- The full board state, scores, bag composition, move history
- Both players’ racks (ground truth, available because it is self-play)
- All legal moves (generated by GADDAG)
- The move selected by the agent
- The game outcome

### 6.2 Belief System Training

The belief encoder/decoder and belief compressor are trained to accurately model the distribution of hidden racks given public information.

**This system is trained independently** of the value network. The belief latent must faithfully represent probability distributions over racks, without being distorted by value estimation objectives. If the value head’s training signal were allowed to leak into the belief compressor, the latent could warp to encode features useful for value prediction but not corresponding to accurate beliefs—polluting the core inference loop. Once trained (or at each checkpoint), the belief system is frozen for the purpose of training the Q/V network.

**Training targets.** The posterior distribution over opponent racks is, in principle, exactly computable. For every possible leave  $\mathbf{r}$  in the unseen tile pool, one can reconstruct the full rack, run GADDAG to enumerate all legal moves, evaluate them with the Q-head to obtain a plausibility score for the observed move, and multiply by the prior. Normalizing yields the exact posterior:

$$P(\mathbf{r} \mid h_t) = \frac{P(\text{move}_{\text{obs}} \mid \mathbf{r}, \text{board}) \cdot P_{\text{prior}}(\mathbf{r})}{\sum_{\mathbf{r}'} P(\text{move}_{\text{obs}} \mid \mathbf{r}', \text{board}) \cdot P_{\text{prior}}(\mathbf{r}')}$$

While enumerating all  $\sim 3.2 \times 10^6$  possible leaves is too expensive for inference, it is feasible *offline* for generating training targets. A spectrum of fidelity/cost tradeoffs is available:

- **Exact:** Brute force over all possible leaves (most expensive, highest fidelity).
- **Approximate:** Sample a large number of leaves (e.g.,  $10^5$ ), evaluate plausibility for each, and use the resulting weighted distribution as a target.
- **Ground truth:** Since training data comes from self-play, the true rack is known. At minimum, the learned posterior should assign reasonable mass to the true rack.

**Training strategy.** Early in training, when the Q-head is weak and cannot provide reliable plausibility scores, ground-truth racks from self-play serve as the primary training signal—the decoder learns to produce samples that include the true rack with high probability. As the Q-head matures, brute-force or approximate posteriors are generated offline for a subset of training positions, and the iterative refinement loop (Section 3.5) is trained to match these targets. The loop learns to approximate in a small number of batches what the brute-force computation achieves over the full leave space.

### 6.3 Q-Head Training

At each training position, we evaluate the V-head on the resulting board state for a randomly selected subset of legal moves (plus the move actually played). The Q-head training target for move  $a_i$  is:

$$y_i = V(s_{a_i}^{\text{post-move}})$$

where  $s_{a_i}^{\text{post-move}}$  is the state *after* tiles are placed on the board but *before* new tiles are drawn from the bag. This ensures the target reflects the quality of the move decision, not the luck of the subsequent draw.

Evaluating only a random subset of legal moves (rather than all  $N$ ) makes training tractable. Over many games, every frequently occurring move receives training signal. The random selection avoids biasing training toward moves that score well by static heuristics.

### 6.4 V-Head Training

The V-head is trained on game outcomes via standard temporal-difference or Monte Carlo return targets. The input includes the belief latent  $\mathbf{z}_t$ , so the V-head benefits from the posterior-refined belief about hidden racks.

### 6.5 Training Loop

The overall training procedure follows a standard self-play improvement cycle:

1. Train the belief system to convergence (or a checkpoint).
2. Freeze the belief system.
3. Generate self-play games using the current Q/V network and belief system.
4. Train the Q-head and V-head on the generated data, where Q-head targets are bootstrapped from V-head evaluations of post-move states.
5. As the V-head improves, Q-head targets improve, which improves move selection, which improves self-play quality.
6. Periodically retrain the belief system on updated self-play data.

## 7 Summary of Design Principles

The architecture is guided by a consistent division of labor:

| Component         | Responsibility                                | Mechanism               |
|-------------------|-----------------------------------------------|-------------------------|
| Neural networks   | Soft, contextual, pattern-based reasoning     | Learned representations |
| GADDAG            | Hard combinatorial constraints, word validity | Exact enumeration       |
| Particle filter   | Probabilistic inference over hidden state     | Bayesian updates        |
| Belief compressor | Bridge between particle space and NN input    | Learned pooling         |

Each component does what it is best at. The neural networks handle context-dependent evaluation and soft pattern recognition. The GADDAG handles exact word finding and move enumeration. The Bayesian machinery handles principled probabilistic inference. Information flows between them through well-defined interfaces: the belief latent, move encodings, and particle sets.

## 8 Future Directions

### 8.1 Search-Derived Knowledge Buffers

A limitation shared by MCTS-based systems across game domains is that insights discovered through deep search are transient—they exist only as backed-up value estimates and are lost once the search tree is discarded. We envision a mechanism analogous to context windows in large language models, but for tree search: a persistent buffer that records position-specific truths discovered during search, which can be read by the neural network for subsequent evaluations within the same search episode.

For example, if GADDAG analysis during simulation discovers that a rare 10-letter word can be formed through disconnected tiles on the current board, this fact is relevant across many branches of the search tree where those board tiles remain intact. Rather than rediscovering this opportunity independently in each branch, the system would record it in a buffer that the value head can consult.

In Scrabble, the practical impact is limited by the stochastic nature of bag draws and hidden information—rack compositions change every turn, so position-specific truths have a short shelf life outside of endgame and pre-endgame play. However, the principle generalizes to other game domains (e.g., complex life-and-death situations in Go) where expensive local calculations produce durable truths relevant across large portions of the search tree.

The current architecture—with its explicit information channels into the neural network (the belief latent, move encodings)—is designed to be extensible to such mechanisms without fundamental restructuring.